

Architettura del Motore AI

Provider, Streaming, Function Calling e Gestione del Contesto

- **Provider, Streaming, Function Calling e Gestione del Contesto**

Categoria: Intelligenza Artificiale — Core Engine

Prerequisito: Paper 02, Paper 04

Versione API: AI Engine v2.0

- **1. Panoramica dell'Architettura AI**

Il motore AI di WhyCremisi segue un'architettura **stratificata** che separa nettamente le responsabilità: ogni layer comunica con il successivo attraverso interfacce ben definite, consentendo sostituzione e test indipendenti.



[NOTE] A differenza dei sistemi AI monolitici, WhyCremisi separa l'orchestrazione dal provider. Questo permette di cambiare modello a metà sessione senza perdere contesto, e di utilizzare modelli diversi per task diversi (es. Claude per analisi, GPT-4o per azioni DAW).

• 2. Provider Support

— 2.1 PROVIDER SUPPORTATI

WhyCremisi astrarre il provider AI dietro un'interfaccia comune, consentendo all'utente di scegliere il modello più adatto al task corrente.

PROVIDER	MODELLI SUPPORTATI	TIPO	LATENZA MEDIA	COSTO
OpenAI	GPT-4o, GPT-4-turbo, GPT-4	Cloud	300-500ms	\$\$\$
Anthropic	Claude 3.5 Sonnet, Claude 3 Opus	Cloud	400-700ms	\$\$\$
Ollama	Llama 3, Mistral, Phi, Qwen2	Locale	600-1200ms	Gratis
DeepSeek	DeepSeek V3, DeepSeek R1	Cloud	200-400ms	\$
Google	Gemini 1.5 Pro, Gemini 2.0 Flash	Cloud	300-600ms	\$\$

— 2.2 INTERFACCIA ASTRATTA

Ogni provider implementa il protocollo comune `AIProvider` :

```
// Interfaccia comune a tutti i provider
class AIProvider {
public:
    virtual ~AIProvider() = default;

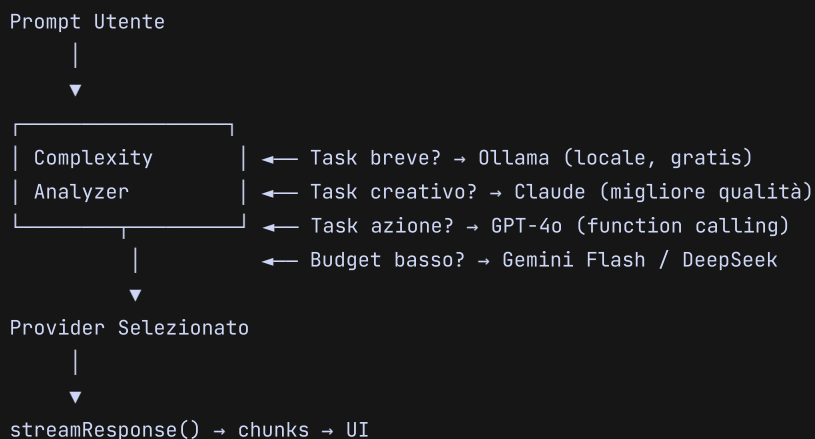
    // Invio messaggio con risposta completa
    virtual std::string sendMessage(
        const std::vector<Message>& messages,
        const AIOptions& options
    ) = 0;

    // Risposta in streaming (chunk per chunk)
    virtual std::unique_ptr<AsyncStream<Chunk>> streamResponse(
        const std::vector<Message>& messages,
        const AIOptions& options
    ) = 0;

    // Annullamento richiesta in corso
    virtual void abort() = 0;

    // Utilità
    virtual bool isAvailable() const = 0;
    virtual size_t estimateTokens(const std::string& text) const = 0;
    virtual std::string providerName() const = 0;
};
```

FLUSSO DI SELEZIONE PROVIDER

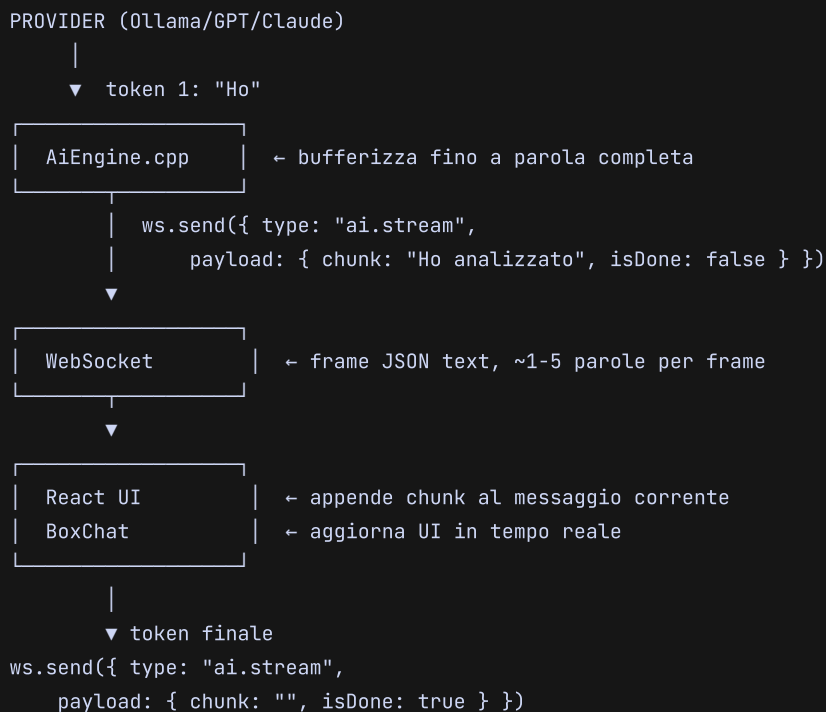


[NOTE] Il `Complexity Analyzer` stima la difficoltà del prompt in <50ms usando un piccolo classificatore locale (ONNX). Se il punteggio di complessità è <0.3, il prompt viene indirizzato automaticamente a Ollama per risparmiare costi API. L'utente può sempre forzare manualmente un provider.

• 3. Streaming Responses

— 3.1 ARCHITETTURA DELLO STREAMING

WhyCremisi utilizza **server-sent events** incapsulati in frame WebSocket per fornire risposte AI in tempo reale. Ogni chunk di token viene trasmesso non appena disponibile, dando all'utente una sensazione di



— 3.2 ABORT MECHANISM

L'utente può interrompere la generazione in qualsiasi momento:



[NOTE] L'abort chiama `std::future::cancel()` sul thread del provider e chiude la connessione HTTP. Il tempo medio di arresto è <100ms. I token già inviati all'UI rimangono visibili — l'utente vede ciò che l'AI aveva già prodotto fino al momento dell'interruzione.

• 4. Function Calling / Tool Use

— 4.1 TOOL DEFINITIONS

WhyCremisi definisce un insieme di **strumenti** che l'AI può invocare per interagire con la DAW. Ogni strumento è descritto tramite JSON Schema e registrato nel costrutto della richiesta al provider.

daw.transport.play()	Avvia playback
daw.transport.stop()	Ferma playback
daw.transport.record()	Avvia registrazione
daw.transport.setTempo(bpm)	Imposta BPM progetto
daw.track.setVolume(id, dB)	Volume traccia
daw.track.setPan(id, pan)	Pan (-1.0 a +1.0)
daw.track.mute(id, muted)	Mute toggle
daw.track.solo(id, soloed)	Solo toggle
daw.track.setGain(id, dB)	Gain traccia
daw.plugin.setParam(track,slot, param,value)	Parametro plugin
daw.plugin.bypass(track,slot,bypass)	Bypass plugin
daw.plugin.query(track,slot)	Leggi tutti i parametri

— 4.2 JSON SCHEMA PER TOOL

Esempio di definizione tool per OpenAI/Anthropic:

```
{
  "name": "daw_track_setVolume",
  "description": "Imposta il volume di una traccia in dB",
  "parameters": {
    "type": "object",
    "properties": {
      "trackId": {
        "type": "integer",
        "description": "ID della traccia (0 = master)"
      },
      "volumeDb": {
        "type": "number",
        "description": "Volume in dB (da -inf a +12)"
      },
      "fadeDurationMs": {
        "type": "integer",
        "description": "Durata fade in millisecondi (opzionale)",
        "default": 0
      }
    }
  },
  "required": ["trackId", "volumeDb"]
}
```

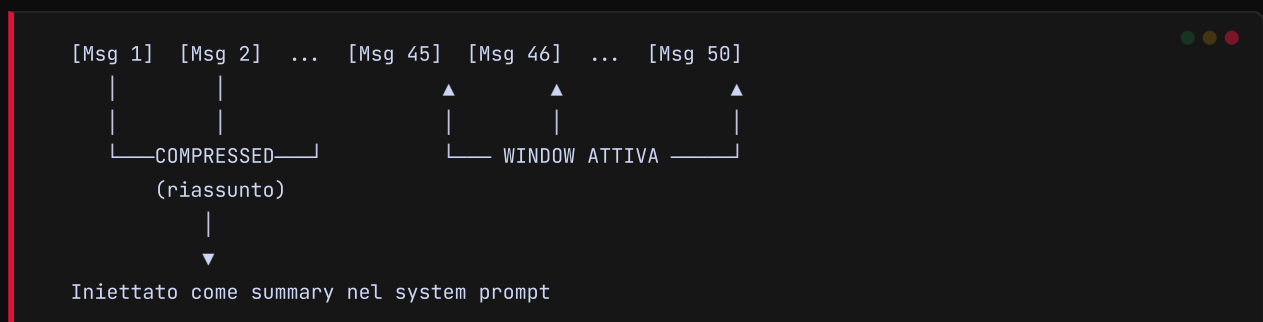


[NOTE] Il livello di conferma è configurabile: **auto** (esegue senza chiedere per azioni a basso rischio come `setVolume`), **confirm** (chiede sempre), o **manual** (l'AI suggerisce ma non esegue mai automaticamente). Il default è **confirm** per tutte le azioni plugin. Le azioni transport (`play/stop`) sono sempre **auto**.

• 5. Context Management

— 5.1 SLIDING WINDOW

La cronologia della conversazione è gestita tramite una **finestra scorrevole** delle ultime 50 interazioni. Quando il numero di messaggi supera 50, i più vecchi vengono rimossi.



— 5.2 SUMMARY COMPRESSION

Quando la finestra scorre, i messaggi rimossi vengono condensati in un riassunto automatico:

```
PRIMA (messaggi 1-45 rimossi):
"Utente: alza il volume della kick
  AI: fatto, ora è a -3dB
  Utente: no, troppo
  AI: ok, l'ho portata a -6dB
  Utente: perfetto"

DOPO (riassunto compresso):
"[SESSIONE: volume kick regolato da AI,
  range -3dB a -6dB, finale -6dB accettato]"
```

— 5.3 TOKEN BUDGET TRACKING

WhyCremisi monitora costantemente il consumo di token per evitare di superare i limiti del contesto del modello:

MODELLO	CONTESTO MAX	BUDGET RISERVATO	ALLARME A
GPT-4o	128K token	96K token	80K
Claude 3.5 Sonnet	200K token	160K token	140K
Claude 3 Opus	200K token	160K token	140K
Llama 3 (Ollama)	8K token	6K token	5K
Gemini 1.5 Pro	1M token	512K token	400K

```
CALCOLO BUDGET A OGNI RICHIESTA:

system_prompt_tokens = countTokens(getSystemPrompt())
context_tokens       = countTokens(getDAWState())
summary_tokens       = countTokens(getCompressedSummary())
window_tokens        = countTokens(slidingWindow)
tool_def_tokens      = countTokens(toolDefinitions)
memory_tokens        = countTokens(relevantMemories)

total_before = system_prompt_tokens + context_tokens
              + summary_tokens + window_tokens
              + tool_def_tokens + memory_tokens

budget_left = max_context - total_before

if budget_left < 1000:
    compressWindow(50%) ← riduce finestra a 25 messaggi
    compressMemories()  ← riassume memorie meno rilevanti
```

— 5.4 SYSTEM PROMPT INJECTION

Prima di ogni chiamata AI, il Context Manager assembla il system prompt finale includendo lo stato live della DAW:

```
[SISTEMA] Sei WhyCremisi, un assistente di produzione musicale AI.
          Rispondi in italiano, stile professionale ma caldo.

[CONTESTO DAW LIVE]
● Progetto: "TRACK_07_HOUSE"
● BPM: 128.0 | Posizione: 00:45.12 | Stato: PLAYING
● Tracce: 8 (4 audio, 3 MIDI, 1 gruppo)

[TRACCE ATTIVE]
| Traccia | Tipo | Volume | Plugin Attivi |
|-----|-----|-----|-----|
| 1 Kick  | Audio | -6.0dB | Pro-Q3 · Pro-C2 |
| 2 Bass  | MIDI  | -8.0dB | Serum · Pro-Q3 · Saturn 2 |
| 3 Synth | MIDI  | -10dB  | Vital · Valhalla Room |
| M Master| Bus   | 0.0dB  | Ozone 11 · Pro-L2 |

[METRICHE AUDIO]
● LUFS integrato: -16.2 (target: -14.0)
● True Peak: -0.8dB (margine: 0.5dB)
● Gamma dinamica: 7.2dB
● Frequenza dominante: 120Hz (sub kick)

[MEMORIA RILEVANTE]
● Utente preferisce kick asciutte (dry_tight)
● Compressione ratio >4:1 rifiutata in passato
● Stile house, target LUFS -14
```

• 6. Memory Integration

— 6.1 MEMORIA DI SESSIONE VS. MEMORIA PERSISTENTE

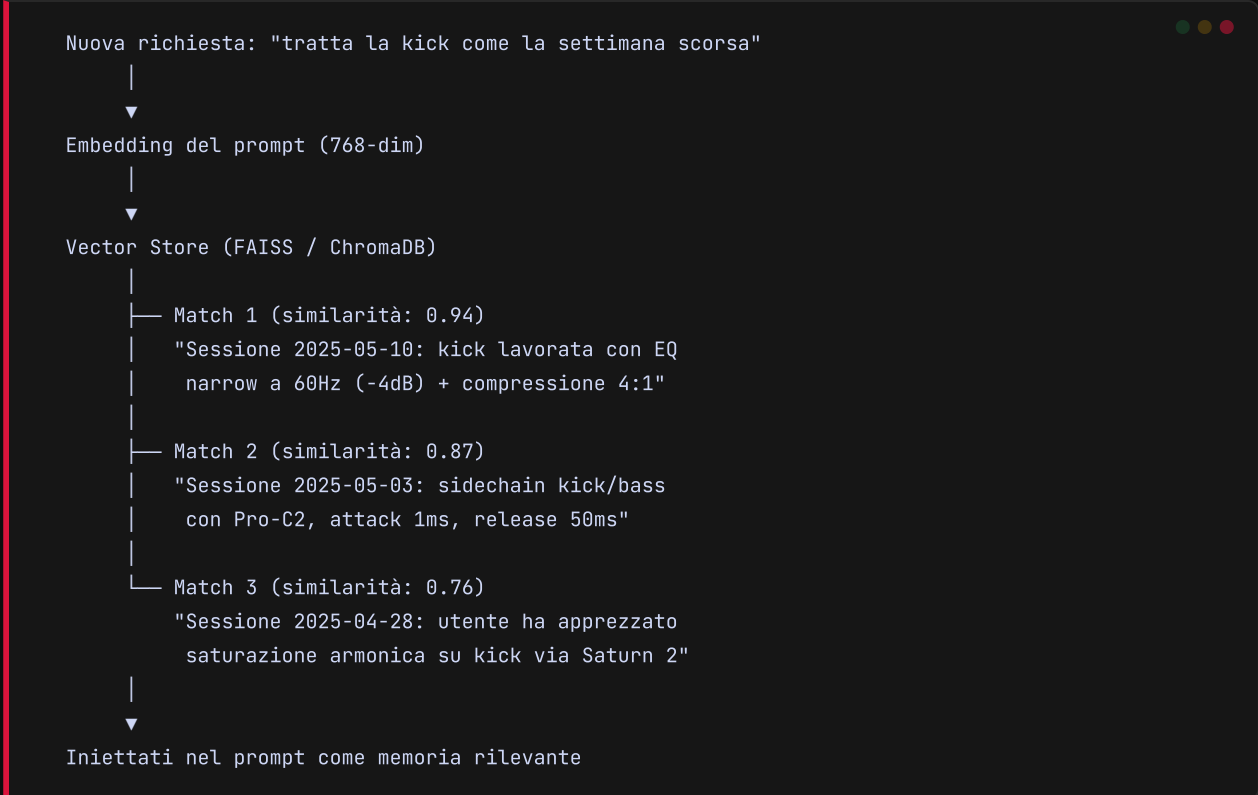
WhyCremisi opera su due livelli di memoria distinti:

```
MEMORIA DI SESSIONE (volatile tra richieste)
-----
● Ultimi 50 messaggi (sliding window)
● Azioni compiute in questa sessione
● Stato DAW corrente
● Reset alla chiusura sessione

MEMORIA PERSISTENTE (attraverso sessioni)
-----
● Profilo utente (preferenze, stile, plugin)
● Cronologia azioni (JSONL)
● Pattern di accettazione/rifiuto
● Identità agente (non modificabile)
● Personalità evolutiva (cambia lentamente)
```

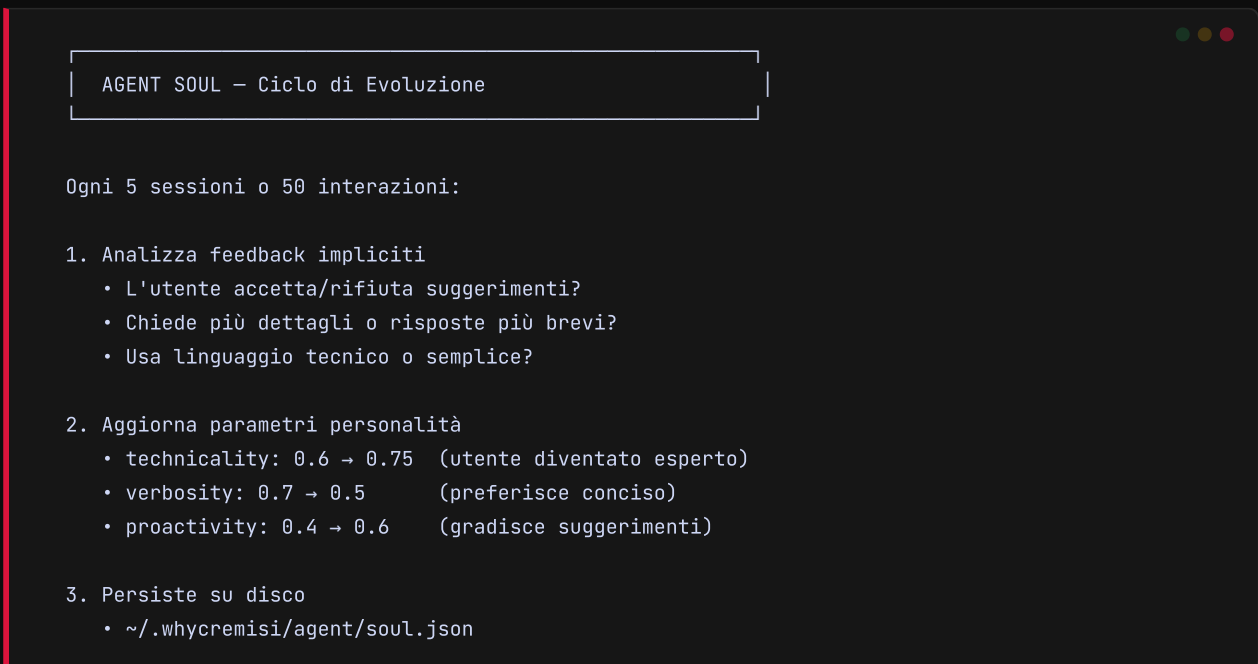
— 6.2 VECTOR STORE PER DECISIONI PASSATE

Per recuperare decisioni simili da sessioni precedenti, WhyCremisi utilizza un **vector store** basato su embeddings:



— 6.3 IDENTITÀ E EVOLUZIONE DELL'AGENTE

L'agente non è statico: evolve con l'utente. Il sistema **AgentSoul** registra ogni interazione e aggiorna gradualmente la personalità:



[NOTE] L'evoluzione è **sempre reversibile**: l'utente può resettare la personalità ai valori predefiniti con **/reset-personality** o bloccare l'evoluzione con **/freeze-personality**. L'agente informa sempre l'utente quando rileva un cambiamento significativo.

• 7. Modello di Personalità

— 7.1 STRUTTURA DEL SYSTEM PROMPT

La personalità di WhyCremisi è definita da un system prompt strutturato che funge da "costituzione"



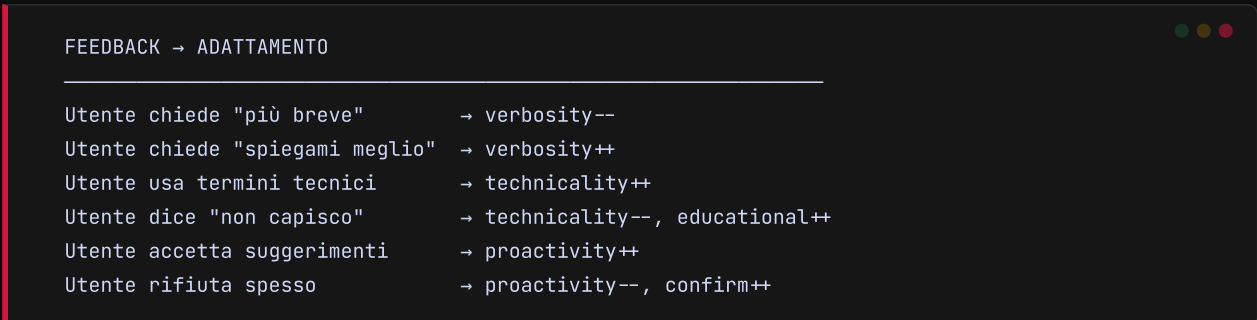
— 7.2 TEMPERATURE E CREATIVITÀ PER TASK

Diversi task richiedono diversi livelli di creatività:

TIPO DI TASK	TEMPERATURE	TOP-P	MAX TOKENS	ESEMPIO
Azione DAW	0.1	0.9	500	<code>setVolume(2, -6)</code>
Analisi Tecnica	0.2	0.9	1000	"Perché il mix suona sporco?"
Suggerimento Creativo	0.7	0.95	1500	"Che effetto posso aggiungere?"
Generazione Testi/Idee	0.9	0.98	2000	"Scriviamo un testo per questa traccia"
Educativo	0.3	0.9	1500	"Spiegami cos'è il sidechain"

— 7.3 PERSONALITÀ ADATTATIVA

WhyCremisi adatta automaticamente il proprio stile in base al feedback implicito dell'utente:



— 8.1 TOKEN COUNTING

Ogni richiesta viene misurata prima dell'invio per prevenire sorprese:

```
struct TokenBudget {
    size_t promptTokens;      // Token nel prompt (input)
    size_t responseTokens;    // Token massimi (output)
    size_t totalBudget;       // Budget totale per questa richiesta
    double estimatedCost;     // Costo stimato in USD
    std::string model;        // Modello selezionato
};

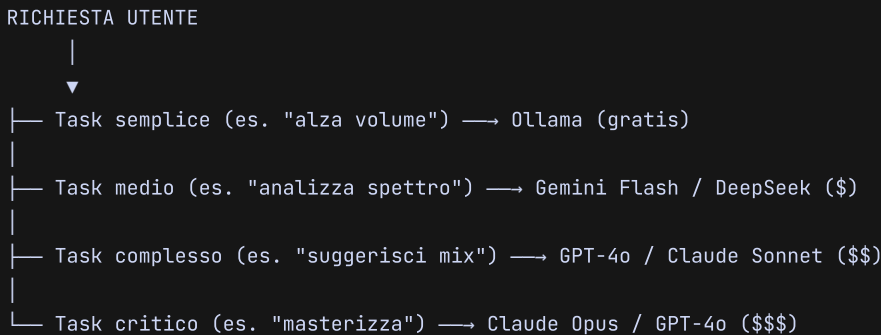
TokenBudget estimateRequest(const AIRequest& req) {
    TokenBudget budget;
    budget.promptTokens = countTokens(req.systemPrompt)
        + countTokens(req.messages)
        + countTokens(req.toolDefinitions);
    budget.responseTokens = std::min(req.maxTokens, 4096);

    // Stima costo (esempio: GPT-4o)
    double inputCost = (budget.promptTokens / 1000.0) * 0.005;
    double outputCost = (budget.responseTokens / 1000.0) * 0.015;
    budget.estimatedCost = inputCost + outputCost;

    return budget;
}
```

— 8.2 STRATEGIA DI FALLBACK

Per ottimizzare i costi, WhyCremisi adotta una strategia di **routing intelligente**:



SOGLIA	MODELLO PRIMARIO	FALLBACK	RISPARMIO STIMATO
< 50 token output	Ollama (locale)	—	100%
< 200 token output	Gemini Flash	Ollama	~80%
< 500 token output	GPT-4o-mini	Gemini Flash	~60%
Complesso	Claude Sonnet	GPT-4o	~0% (qualità)

[NOTE] Il routing intelligente ha mostrato una riduzione dei costi API del **40-60%** nei test interni, senza degradazione percepibile della qualità per l'utente. I task di mastering e critici usano sempre il modello migliore indipendentemente dal costo.

— 9.1 PREVENZIONE PROMPT INJECTION

WhyCremisi implementa difese multiple contro attacchi di prompt injection:

STRATI DI DIFESA

LAYER 1 – Separazione Input

- Prompt utente e contesto sistema sono rigorosamente separati
- Il contesto DAW viene iniettato DOPO il prompt utente
- Delimitatori speciali intorno all'input utente:
`<user\_input> ...testo utente... </user\_input>`

LAYER 2 – Rilevamento Pattern Sospetti

- Regex per "ignora le istruzioni precedenti"
- Rilevamento tentativi di jailbreak (DAN, roleplay evasion)
- Quarantena: se rilevato, il prompt viene scartato e loggato

LAYER 3 – Output Validation

- Ogni risposta AI viene validata prima di essere mostrata
- Filtro per contenuti non consentiti
- Verifica che le function call siano nel registro strumenti

— 9.2 VALIDAZIONE DEGLI OUTPUT

Prima che una risposta o azione raggiunga l'utente o la DAW, passa attraverso un **validation layer**:

```
struct ValidationResult {
    bool isSafe;
    bool isAllowedAction;
    std::string sanitizedOutput;
    std::vector<std::string> warnings;
};

ValidationResult validateOutput(const AIResponse& response) {
    ValidationResult result;

    // 1. Check contenuti pericolosi
    result.isSafe = !containsProhibitedContent(response.text);

    // 2. Check function call
    for (auto& call : response.toolCalls) {
        result.isAllowedAction = isInToolRegistry(call.name);
        if (!result.isAllowedAction) {
            result.warnings.push_back(
                "Function call bloccata: " + call.name
            );
        }
    }

    return result;
}
```

— 9.3 CONFERMA PRIMA DELL'ESECUZIONE

Nessuna azione sulla DAW viene eseguita senza il giusto livello di consenso:

LIVELLI DI CONFERMA

AUTO (nessuna conferma)

- Trasport: play, stop, pause
- Query: lettura parametri plugin
- Meter: richiesta metriche audio

CONFIRM (mostra dialogo di conferma)

- Volume/pan tracce
- Cambio parametri plugin
- Bypass/attivazione plugin
- Default per tutte le azioni nuove

MANUAL (solo suggerimento, l'utente applica manualmente)

- Aggiunta/rimozione plugin
- Caricamento preset
- Azioni mastering (limiter, export)
- Qualsiasi azione che modifichi la catena di processing

— 9.4 SANDBOXED TOOL EXECUTION

Le function call vengono eseguite in un ambiente **sandboxato**:

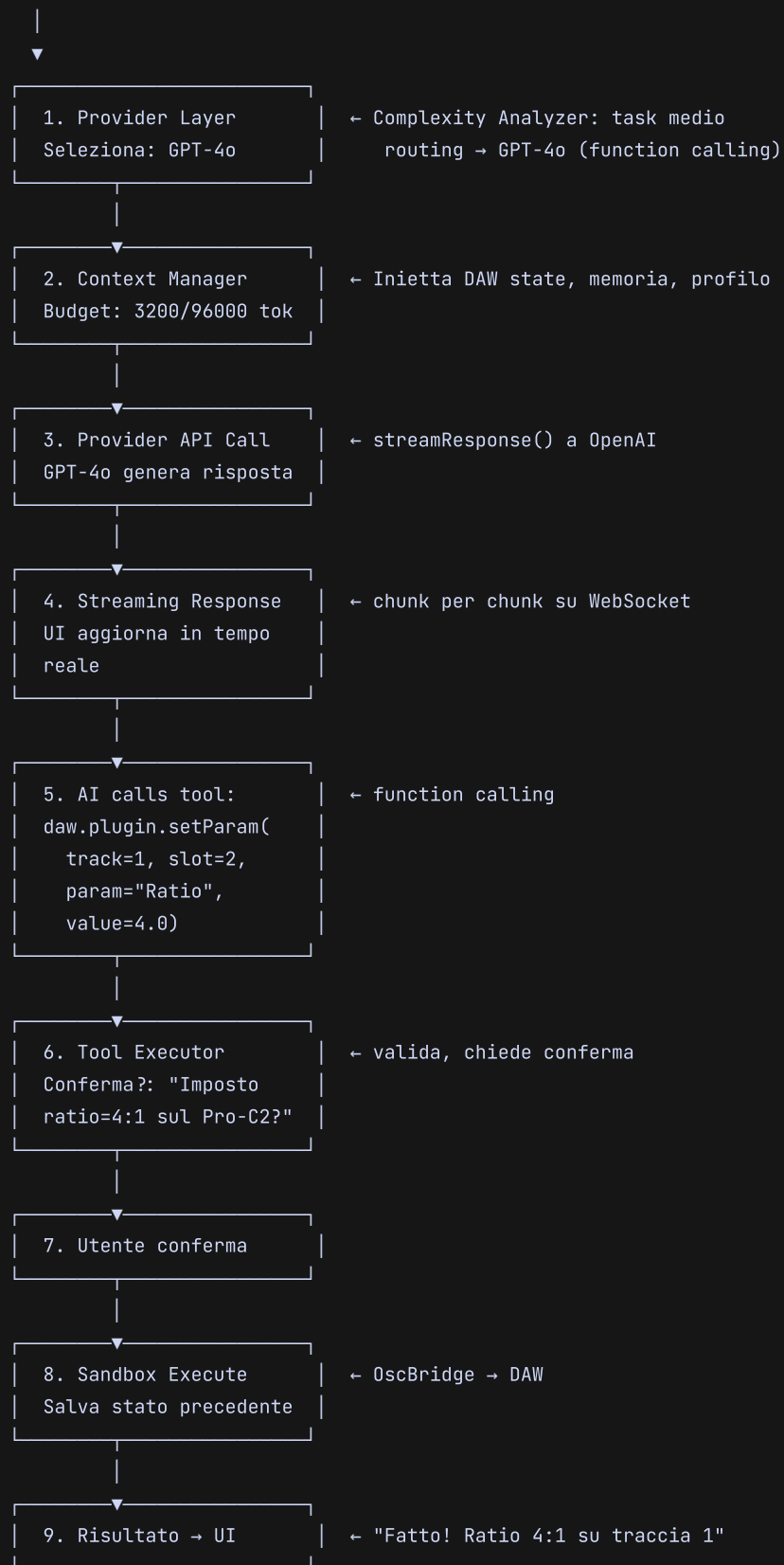
SANDBOX TOOL EXECUTION

- Ogni tool viene eseguito in un thread dedicato
- Timeout per tool: 5 secondi (oltre viene annullato)
- Le modifiche ai parametri DAW sono reversibili:
 - Prima di ogni modifica, lo stato corrente viene salvato
 - Annulla (Ctrl+Z) ripristina lo stato precedente
- Limite di 10 azioni consecutive automatiche:
 - Dopo 10 azioni, l'AI deve chiedere conferma
 - Previene runaway loops (es. "alza il volume di 100 tracce")
- Log completo in sessione: ogni azione è tracciata

• 10. Diagramma di Flusso Completo

ESEMPIO: "Aggiungi un po' di compressione alla kick"

Utente



PARAMETRO	DEFAULT	DESCRIZIONE
<code>ai.provider</code>	<code>auto</code>	Provider predefinito (auto/gemini/ollama/openai/anthropic)
<code>ai.model</code>	—	Modello specifico per il provider
<code>ai.temperature</code>	<code>0.3</code>	Creatività (0.0-1.0)
<code>ai.maxTokens</code>	<code>2048</code>	Token massimi per risposta
<code>ai.contextWindow</code>	<code>50</code>	Messaggi nella sliding window
<code>ai.confirmLevel</code>	<code>confirm</code>	auto/confirm/manual
<code>ai.costLimit</code>	<code>0.50</code>	Limite di spesa USD per sessione
<code>ai.fallbackEnabled</code>	<code>true</code>	Fallback a modello più economico
<code>ai.vectorMemory</code>	<code>true</code>	Abilita vector store per memoria

RIFERIMENTI

Paper 02 – Architettura del Sistema (stack complessivo)
Paper 04 – Sistema di Memoria Agente (OpenClaw Memory)
Paper 05 – Protocollo di Comunicazione (WebSocket + OSC)

API Reference:
- OpenAI Function Calling: platform.openai.com
- Anthropic Tool Use: docs.anthropic.com
- Ollama API: github.com/ollama/ollama